

In [1]: `import pandas as pd`

```
# Load dataset
df = pd.read_csv('fraud_detection_dataset.csv')

# Show basic info
print(" Columns:\n", df.columns.tolist())
print("\n Shape:", df.shape)
print("\n Missing values:\n", df.isnull().sum())
print("\n Data types:\n", df.dtypes)
df.head()
```

Columns:

```
['Transaction_ID', 'User_ID', 'Transaction_Amount', 'Transaction_Type', 'Timestamp', 'Account_Balance', 'Device_Type', 'Location', 'Merchant_Category', 'IP_Address_Flag', 'Previous_Fraudulent_Activity', 'Daily_Transaction_Count', 'Avg_Transaction_Amount_7d', 'Failed_Transaction_Count_7d', 'Card_Type', 'Card_Age', 'Transaction_Distance', 'Authentication_Method', 'Risk_Score', 'Is_Weekend', 'Fraud_Label']
```

Shape: (50000, 21)

Missing values:

Transaction_ID	0
User_ID	0
Transaction_Amount	0
Transaction_Type	0
Timestamp	0
Account_Balance	0
Device_Type	0
Location	0
Merchant_Category	0
IP_Address_Flag	0
Previous_Fraudulent_Activity	0
Daily_Transaction_Count	0
Avg_Transaction_Amount_7d	0
Failed_Transaction_Count_7d	0
Card_Type	0
Card_Age	0
Transaction_Distance	0
Authentication_Method	0
Risk_Score	0
Is_Weekend	0
Fraud_Label	0

dtype: int64

Data types:

Transaction_ID	object
User_ID	object
Transaction_Amount	float64
Transaction_Type	object

```

Timestamp                object
Account_Balance           float64
Device_Type               object
Location                  object
Merchant_Category         object
IP_Address_Flag           int64
Previous_Fraudulent_Activity int64
Daily_Transaction_Count   int64
Avg_Transaction_Amount_7d float64
Failed_Transaction_Count_7d int64
Card_Type                 object
Card_Age                  int64
Transaction_Distance      float64
Authentication_Method     object
Risk_Score                float64
Is_Weekend                int64
Fraud_Label               int64
dtype: object

```

```

Out[1]:
   Transaction_ID  User_ID  Transaction_Amount  Transaction_Type  Timestamp
0      TXN_33553  USER_1834             39.79                POS  2023-14 19:30
1      TXN_9427  USER_7875              1.19        Bank Transfer  2023-04:01
2      TXN_199  USER_2734             28.96                Online  2023-15:25
3      TXN_12447  USER_2617            254.32        ATM Withdrawal  2023-00:31
4      TXN_39489  USER_2014             31.28                POS  2023-1' 23:44

```

5 rows × 21 columns

```

In [2]: # Convert 'Timestamp' to datetime
df['Timestamp'] = pd.to_datetime(df['Timestamp'])

# Drop ID columns (not useful for modeling)
df.drop(columns=['Transaction_ID', 'User_ID'], inplace=True)

# Confirm changes
print("Data cleaned. New shape:", df.shape)
print("\nColumn types after cleaning:\n", df.dtypes)

```

Data cleaned. New shape: (50000, 19)

Column types after cleaning:

Transaction_Amount	float64
Transaction_Type	object
Timestamp	datetime64[ns]
Account_Balance	float64
Device_Type	object
Location	object
Merchant_Category	object
IP_Address_Flag	int64
Previous_Fraudulent_Activity	int64
Daily_Transaction_Count	int64
Avg_Transaction_Amount_7d	float64
Failed_Transaction_Count_7d	int64
Card_Type	object
Card_Age	int64
Transaction_Distance	float64
Authentication_Method	object
Risk_Score	float64
Is_Weekend	int64
Fraud_Label	int64
dtype:	object

```
In [3]: reduced_df = df.drop(columns=['Timestamp', 'Location'])
```

```
In [4]: reduced_df = reduced_df.drop(columns=['Transaction_ID', 'User_ID'])
```

```
-----
-----
KeyError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 reduced_df = reduced_df.drop(columns=[
    ],
    )

File ~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/pandas/core/frame.py:5581, in DataFrame.drop(self, labels, axis, index, columns, level, inplace, errors)
    5433 def drop(
    5434     self,
    5435     labels: IndexLabel | None = None,
    (... ) 5442     errors: IgnoreRaise = "raise",
    5443 ) -> DataFrame | None:
    5444     """
    5445     Drop specified labels from rows or columns.
    5446
    (... ) 5579             weight 1.0      0.8
    5580     """
-> 5581     return super().drop(
    5582         labels=labels,
    5583         axis=axis,
```

```

5584         index=index,
5585         columns=columns,
5586         level=level,
5587         inplace=inplace,
5588         errors=errors,
5589     )

```

File ~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/pandas/core/generic.py:4788, in NDFrame.drop(self, labels, axis, index, columns, level, inplace, errors)

```

4786 for axis, labels in axes.items():
4787     if labels is not None:
-> 4788         obj = obj._drop_axis(labels, axis, level=level, errors=
errors)
4790 if inplace:
4791     self._update_inplace(obj)

```

File ~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/pandas/core/generic.py:4830, in NDFrame._drop_axis(self, labels, axis, level, errors, only_slice)

```

4828     new_axis = axis.drop(labels, level=level, errors=errors)
s)
4829     else:
-> 4830         new_axis = axis.drop(labels, errors=errors)
4831     indexer = axis.get_indexer(new_axis)
4833 # Case for non-unique axis
4834 else:

```

File ~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/pandas/core/indexes/base.py:7070, in Index.drop(self, labels, errors)

```

7068 if mask.any():
7069     if errors != "ignore":
-> 7070         raise KeyError(f"{labels[mask].tolist()} not found in a
xis")
7071     indexer = indexer[~mask]
7072 return self.delete(indexer)

```

KeyError: "['Transaction_ID', 'User_ID'] not found in axis"

```
In [5]: reduced_df.select_dtypes(include='object').columns
```

```
Out[5]: Index(['Transaction_Type', 'Device_Type', 'Merchant_Category', 'Card_
Type',
             'Authentication_Method'],
             dtype='object')
```

```
In [8]: from sklearn.preprocessing import OneHotEncoder

categorical_cols = reduced_df.select_dtypes(include='object').columns

encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
encoded = encoder.fit_transform(reduced_df[categorical_cols])
encoded_df = pd.DataFrame(encoded, columns=encoder.get_feature_names_o
```

```
numeric_df = reduced_df.drop(columns=categorical_cols).reset_index(drop=True)
full_encoded_df = pd.concat([numeric_df, encoded_df], axis=1)
```

In [9]: `from sklearn.preprocessing import MinMaxScaler`

```
X = full_encoded_df.drop(columns=['Fraud_Label'])
y = full_encoded_df['Fraud_Label']

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

normalized_df = pd.DataFrame(X_scaled, columns=X.columns)
normalized_df['Fraud_Label'] = y.values # Add the target column back
```

In [10]: `print(normalized_df.head())`

```
print(normalized_df.describe())
```

	Transaction_Amount	Account_Balance	IP_Address_Flag	\
0	0.033889	0.931806	0.0	
1	0.001014	0.756044	0.0	
2	0.024665	0.010940	0.0	
3	0.216601	0.766918	0.0	
4	0.026641	0.923178	0.0	

	Previous_Fraudulent_Activity	Daily_Transaction_Count	\
0	0.0	0.461538	
1	0.0	0.923077	
2	0.0	1.000000	
3	0.0	0.538462	
4	1.0	1.000000	

	Avg_Transaction_Amount_7d	Failed_Transaction_Count_7d	Card_Age	\
0	0.872714	0.75	0.268908	
1	0.956653	1.00	0.777311	
2	0.081653	1.00	0.945378	
3	0.352000	1.00	0.315126	
4	0.650388	1.00	0.584034	

	Transaction_Distance	Risk_Score	...	Merchant_Category_Travel	\
0	0.176595	0.849385	...		1.0
1	0.440650	0.095810	...		0.0
2	0.381832	0.839984	...		0.0
3	0.262339	0.793479	...		0.0
4	0.193358	0.381838	...		0.0

	Card_Type_Amex	Card_Type_Discover	Card_Type_Mastercard	Card_Type_Visa	\
0	1.0	0.0	0.0		
0.0					
1	0.0	0.0	1.0		

0.0			
2	0.0	0.0	0.0
1.0			
3	0.0	0.0	0.0
1.0			
4	0.0	0.0	1.0
0.0			

	Authentication_Method_Biometric	Authentication_Method_OTP	\
0	1.0	0.0	
1	0.0	0.0	
2	1.0	0.0	
3	0.0	1.0	
4	0.0	0.0	

	Authentication_Method_PIN	Authentication_Method_Password	Fraud_Label
0	0.0	0.0	
0			
1	0.0	1.0	
1			
2	0.0	0.0	
1			
3	0.0	0.0	
1			
4	0.0	1.0	
1			

[5 rows x 32 columns]

	Transaction_Amount	Account_Balance	IP_Address_Flag	\
count	50000.000000	50000.000000	50000.000000	
mean	0.084667	0.500449	0.05020	
std	0.084051	0.289056	0.21836	
min	0.000000	0.000000	0.000000	
25%	0.024424	0.249810	0.000000	
50%	0.059329	0.501357	0.000000	
75%	0.118259	0.749912	0.000000	
max	1.000000	1.000000	1.000000	

	Previous_Fraudulent_Activity	Daily_Transaction_Count	\
count	50000.000000	50000.000000	
mean	0.098400	0.498865	
std	0.297858	0.310741	
min	0.000000	0.000000	
25%	0.000000	0.230769	
50%	0.000000	0.461538	
75%	0.000000	0.769231	
max	1.000000	1.000000	

	Avg_Transaction_Amount_7d	Failed_Transaction_Count_7d	Card_Age	\
count	50000.000000	50000.000000	50000.00	

0000			
mean	0.500555	0.500885	0.50
0000			
std	0.288535	0.353568	0.28
9856			
min	0.000000	0.000000	0.00
0000			
25%	0.249158	0.250000	0.24
7899			
50%	0.502214	0.500000	0.50
0000			
75%	0.751087	0.750000	0.75
2101			
max	1.000000	1.000000	1.00
0000			

	Transaction_Distance	Risk_Score	...	Merchant_Category_Trave
l \				
count	50000.000000	50000.000000	...	50000.000000
0				
mean	0.499815	0.501506	...	0.20030
0				
std	0.288421	0.287803	...	0.40022
9				
min	0.000000	0.000000	...	0.00000
0				
25%	0.251266	0.253925	...	0.00000
0				
50%	0.498139	0.502200	...	0.00000
0				
75%	0.749277	0.749500	...	0.00000
0				
max	1.000000	1.000000	...	1.00000
0				

	Card_Type_Amex	Card_Type_Discover	Card_Type_Mastercard	\
count	50000.000000	50000.000000	50000.000000	
mean	0.248380	0.246560	0.253860	
std	0.432078	0.431013	0.435223	
min	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	
50%	0.000000	0.000000	0.000000	
75%	0.000000	0.000000	1.000000	
max	1.000000	1.000000	1.000000	

	Card_Type_Visa	Authentication_Method_Biometric	\
count	50000.000000	50000.000000	
mean	0.251200	0.251820	
std	0.433708	0.434063	
min	0.000000	0.000000	
25%	0.000000	0.000000	
50%	0.000000	0.000000	

75%	1.000000	1.000000
max	1.000000	1.000000

	Authentication_Method_OTP	Authentication_Method_PIN \
count	50000.000000	50000.000000
mean	0.247320	0.251720
std	0.431459	0.434006
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	1.000000
max	1.000000	1.000000

	Authentication_Method_Password	Fraud_Label
count	50000.000000	50000.000000
mean	0.249140	0.321340
std	0.432519	0.466996
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	0.000000	1.000000
max	1.000000	1.000000

[8 rows x 32 columns]

```
In [11]: columns_to_discretize = [
    'Transaction_Amount',
    'Account_Balance',
    'Risk_Score',
    'Transaction_Distance'
]

for col in columns_to_discretize:
    full_encoded_df[f'{col}_binned'] = pd.cut(
        full_encoded_df[col],
        bins=3,
        labels=['Low', 'Medium', 'High']
    )

print("Discretization sample:")
print(full_encoded_df[[f'{col}_binned' for col in columns_to_discretize]])
```


Discretization sample:

	Transaction_Amount_binned	Account_Balance_binned	Risk_Score_binned	\
0	Low	High	High	
1	Low	High	Low	
2	Low	Low	High	
3	Low	High	High	
4	Low	High	Medium	

	Transaction_Distance_binned	Fraud_Label
0	Low	0
1	Medium	1
2	Medium	1
3	Low	1
4	Low	1

```
In [12]: from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

X = full_encoded_df.drop(columns=['Fraud_Label'])
y = full_encoded_df['Fraud_Label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
In [13]: rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
```

```
-----
ValueError                                Traceback (most recent call last)
/var/folders/sv/htfpddmj4h791yr31hgl24p40000gn/T/ipykernel_63249/103030087.py in ?()
      1 rf = RandomForestClassifier(n_estimators=100, random_state=42)
----> 2 rf.fit(X_train, y_train)
      3
      4 y_pred = rf.predict(X_test)

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/base.py in ?(estimator, *args, **kwargs)
    1385         skip_parameter_validation=(
    1386             prefer_skip_nested_validation or global_skip_validation
    1387         )
    1388     ):
-> 1389         return fit_method(estimator, *args, **kwargs)
```

```

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/ensemble/_forest.py in ?(self, X, y, sample_weight)
    356         # Validate or convert input data
    357         if issparse(y):
    358             raise ValueError("sparse multilabel-indicator for y
is not supported.")
    359
--> 360         X, y = validate_data(
    361             self,
    362             X,
    363             y,

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/uti
ls/validation.py in ?(_estimator, X, y, reset, validate_separately, ski
p_check_array, **check_params)
    2957         if "estimator" not in check_y_params:
    2958             check_y_params = {**default_check_params, **che
ck_y_params}
    2959             y = check_array(y, input_name="y", **check_y_param
s)
    2960         else:
-> 2961             X, y = check_X_y(X, y, **check_params)
    2962             out = X, y
    2963
    2964         if not no_val_X and check_params.get("ensure_2d", True):

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/uti
ls/validation.py in ?(X, y, accept_sparse, accept_large_sparse, dtype,
order, copy, force_writeable, force_all_finite, ensure_all_finite, ensu
re_2d, allow_nd, multi_output, ensure_min_samples, ensure_min_features,
y_numeric, estimator)
    1366         )
    1367
    1368         ensure_all_finite = _deprecate_force_all_finite(force_all_f
inite, ensure_all_finite)
    1369
-> 1370         X = check_array(
    1371             X,
    1372             accept_sparse=accept_sparse,
    1373             accept_large_sparse=accept_large_sparse,

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/uti
ls/validation.py in ?(array, accept_sparse, accept_large_sparse, dtype,
order, copy, force_writeable, force_all_finite, ensure_all_finite, ensu
re_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_fe
atures, estimator, input_name)
    1052         )
    1053         array = xp.astype(array, dtype, copy=False)
    1054         else:
    1055             array = _asarray_with_order(array, order=or
der, dtype=dtype, xp=xp)
-> 1056         except ComplexWarning as complex_warning:

```

```

1057         raise ValueError(
1058             "Complex data not supported\n{}\n".format(a
rray)
1059         ) from complex_warning

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/uti
ls/_array_api.py in ?(array, dtype, order, copy, xp, device)
    835     # Use NumPy API to support order
    836     if copy is True:
    837         array = numpy.array(array, order=order, dtype=dtyp
e)
    838     else:
--> 839         array = numpy.asarray(array, order=order, dtype=dty
pe)
    840
    841     # At this point array is a NumPy ndarray. We convert it
to an array
    842     # container that is consistent with the input's namespa
ce.

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/pandas/cor
e/generic.py in ?(self, dtype, copy)
    2149     def __array__(
    2150         self, dtype: npt.DTypeLike | None = None, copy: bool_t
| None = None
    2151     ) -> np.ndarray:
    2152         values = self._values
-> 2153         arr = np.asarray(values, dtype=dtype)
    2154         if (
    2155             astype_is_view(values.dtype, arr.dtype)
    2156             and using_copy_on_write()
ValueError: could not convert string to float: 'Low'

```

```

In [14]: X_clean = full_encoded_df.drop(columns=[
        'Transaction_Amount_binned',
        'Account_Balance_binned',
        'Risk_Score_binned',
        'Transaction_Distance_binned'
    ])

X = X_clean.drop(columns=['Fraud_Label'])
y = X_clean['Fraud_Label']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

```

```

In [15]: rf = RandomForestClassifier(n_estimators=100, random_state=42)
        rf.fit(X_train, y_train)

```

```
y_pred = rf.predict(X_test)
```

```
In [16]: from sklearn.metrics import classification_report, confusion_matrix

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Confusion Matrix:

```
[[6787   0]
 [   0 3213]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6787
1	1.00	1.00	1.00	3213
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

```
In [17]: import pandas as pd
import matplotlib.pyplot as plt

importances = pd.Series(rf.feature_importances_, index=X.columns)
importances = importances.sort_values(ascending=False)

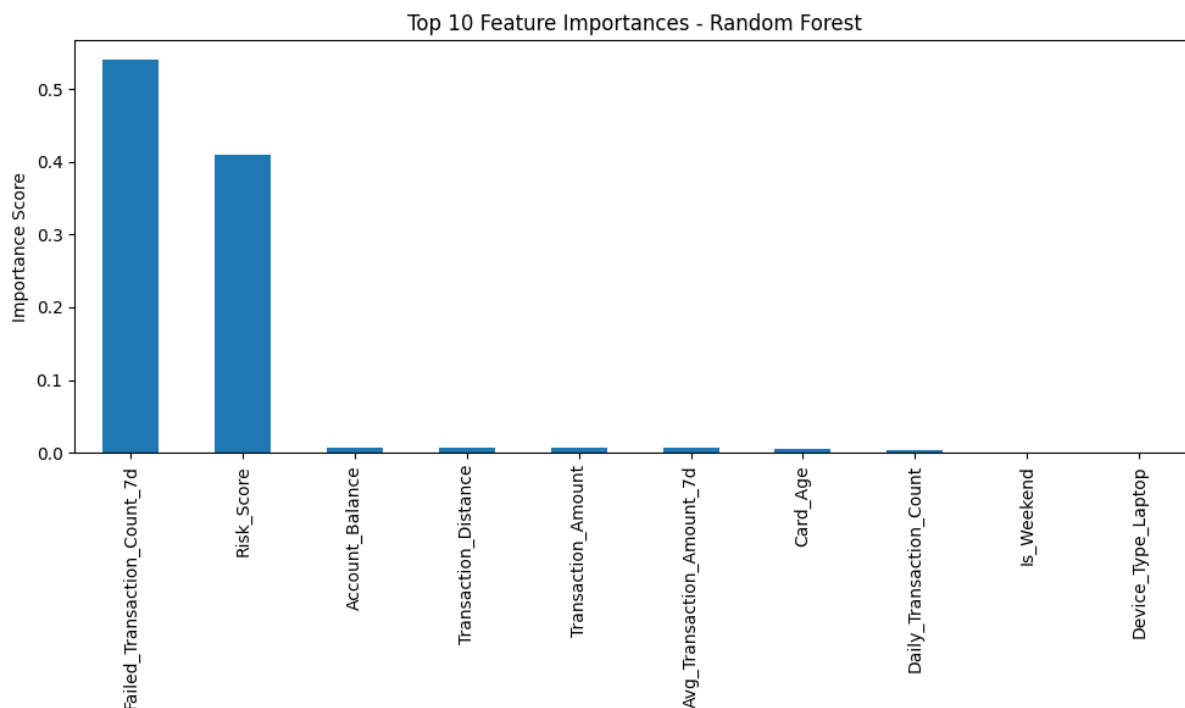
print("\nTop 10 Important Features:")
print(importances.head(10))

plt.figure(figsize=(10,6))
importances.head(10).plot(kind='bar')
plt.title("Top 10 Feature Importances - Random Forest")
plt.ylabel("Importance Score")
plt.tight_layout()
plt.show()
```

Top 10 Important Features:

Failed_Transaction_Count_7d	0.539656
Risk_Score	0.409351
Account_Balance	0.006856
Transaction_Distance	0.006810
Transaction_Amount	0.006756
Avg_Transaction_Amount_7d	0.006676
Card_Age	0.006003
Daily_Transaction_Count	0.003514
Is_Weekend	0.000917
Device_Type_Laptop	0.000692

dtype: float64



```
In [18]: from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix

xgb = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb.fit(X_train, y_train)

y_pred_xgb = xgb.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_xgb))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_xgb))
```

Confusion Matrix:

```
[[6783    4]
 [    4 3209]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6787
1	1.00	1.00	1.00	3213
accuracy			1.00	10000
macro avg	1.00	1.00	1.00	10000
weighted avg	1.00	1.00	1.00	10000

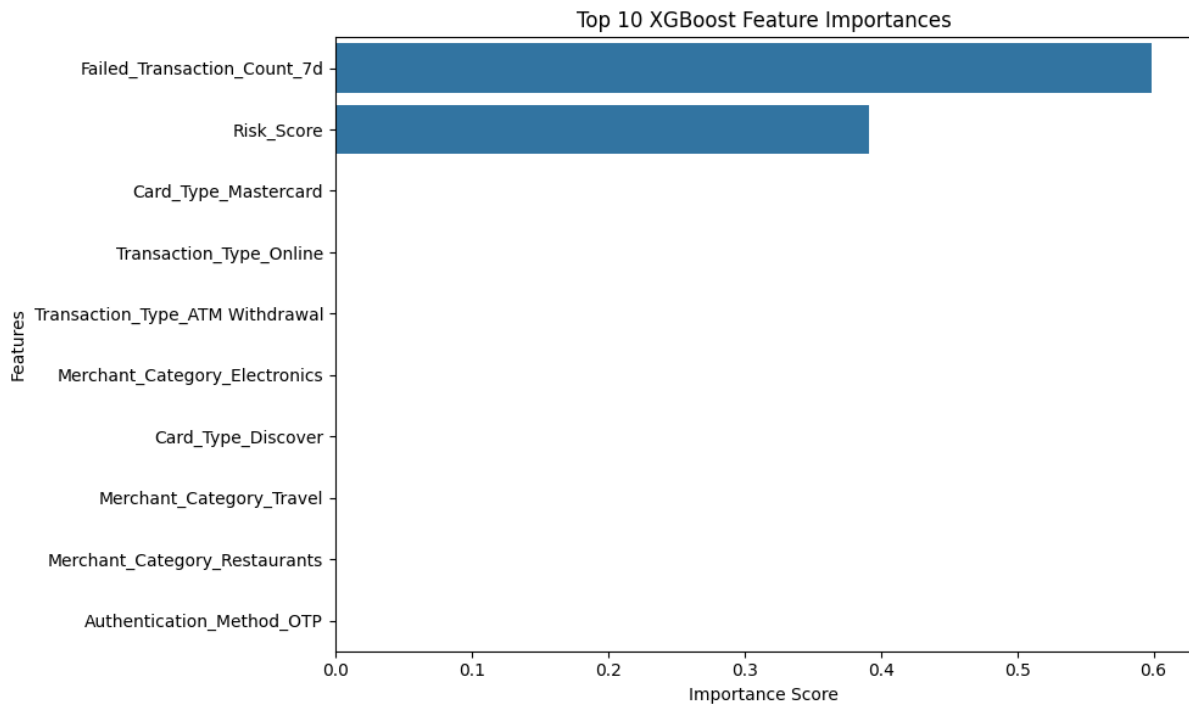
```
/Users/abdul-salembeibitkhan/Desktop/COMP_4353_Final/venv/lib/python3.1
3/site-packages/xgboost/training.py:183: UserWarning: [10:52:58] WARNIN
G: /Users/runner/work/xgboost/xgboost/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```
In [19]: import matplotlib.pyplot as plt
import seaborn as sns

importances = pd.Series(xgb.feature_importances_, index=X_train.column

plt.figure(figsize=(10, 6))
sns.barplot(x=importances[:10], y=importances.index[:10])
plt.title("Top 10 XGBoost Feature Importances")
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
```



```
In [20]: final_df_model_ready = full_encoded_df.copy()
```

```
In [21]: from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

X = final_df_model_ready.drop(columns=['Fraud_Label'])
y = final_df_model_ready['Fraud_Label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.

log_reg = LogisticRegression(max_iter=1000, solver='liblinear')
log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cma
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```

-----
ValueError                                Traceback (most recent call last)
/var/folders/sv/htfpddmj4h791yr31hgl24p40000gn/T/ipykernel_63249/3557174907.py in ?()
     10
     11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
     12
     13 log_reg = LogisticRegression(max_iter=1000, solver='liblinear')
--> 14 log_reg.fit(X_train, y_train)
     15
     16 y_pred = log_reg.predict(X_test)
     17

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/base.py in ?(estimator, *args, **kwargs)
    1385         skip_parameter_validation=(
    1386             prefer_skip_nested_validation or global_skip_validation
    1387         )
    1388     ):
-> 1389         return fit_method(estimator, *args, **kwargs)

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/linear_model/_logistic.py in ?(self, X, y, sample_weight)
    1218         _dtype = np.float64
    1219     else:
    1220         _dtype = [np.float64, np.float32]
    1221
-> 1222     X, y = validate_data(
    1223         self,
    1224         X,
    1225         y,

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/utils/validation.py in ?(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
    2957         if "estimator" not in check_y_params:
    2958             check_y_params = {**default_check_params, **check_y_params}
    2959         y = check_array(y, input_name="y", **check_y_params)
    2960     else:
-> 2961         X, y = check_X_y(X, y, **check_params)
    2962         out = X, y
    2963
    2964     if not no_val_X and check_params.get("ensure_2d", True):

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/utils/validation.py in ?(X, y, accept_sparse, accept_large_sparse, dtype,

```



```

order, copy, force_writeable, force_all_finite, ensure_all_finite, ensu
re_2d, allow_nd, multi_output, ensure_min_samples, ensure_min_features,
y_numeric, estimator)
1366         )
1367
1368     ensure_all_finite = _deprecate_force_all_finite(force_all_f
inite, ensure_all_finite)
1369
-> 1370     X = check_array(
1371         X,
1372         accept_sparse=accept_sparse,
1373         accept_large_sparse=accept_large_sparse,

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/uti
ls/validation.py in ?(array, accept_sparse, accept_large_sparse, dtype,
order, copy, force_writeable, force_all_finite, ensure_all_finite, ensu
re_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_fe
atures, estimator, input_name)
1052         )
1053         array = xp.astype(array, dtype, copy=False)
1054     else:
1055         array = _asarray_with_order(array, order=or
der, dtype=dtype, xp=xp)
-> 1056     except ComplexWarning as complex_warning:
1057         raise ValueError(
1058             "Complex data not supported\n{}\n".format(a
rray)
1059         ) from complex_warning

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/sklearn/uti
ls/_array_api.py in ?(array, dtype, order, copy, xp, device)
835     # Use NumPy API to support order
836     if copy is True:
837         array = numpy.array(array, order=order, dtype=typ
e)
838     else:
--> 839         array = numpy.asarray(array, order=order, dtype=typ
pe)
840
841     # At this point array is a NumPy ndarray. We convert it
to an array
842     # container that is consistent with the input's namespa
ce.

~/Desktop/COMP_4353_Final/venv/lib/python3.13/site-packages/pandas/cor
e/generic.py in ?(self, dtype, copy)
2149     def __array__(
2150         self, dtype: npt.DTypeLike | None = None, copy: bool_t
| None = None
2151     ) -> np.ndarray:
2152         values = self._values
-> 2153         arr = np.asarray(values, dtype=dtype)

```

```

2154         if (
2155             astype_is_view(values.dtype, arr.dtype)
2156             and using_copy_on_write()

```

ValueError: could not convert string to float: 'Low'

```

In [22]: from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

logistic_df = full_encoded_df.select_dtypes(include=['number']).copy()

X = logistic_df.drop(columns=['Fraud_Label'])
y = logistic_df['Fraud_Label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.

log_reg = LogisticRegression(max_iter=1000, solver='liblinear')
log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cma
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```

Confusion Matrix:

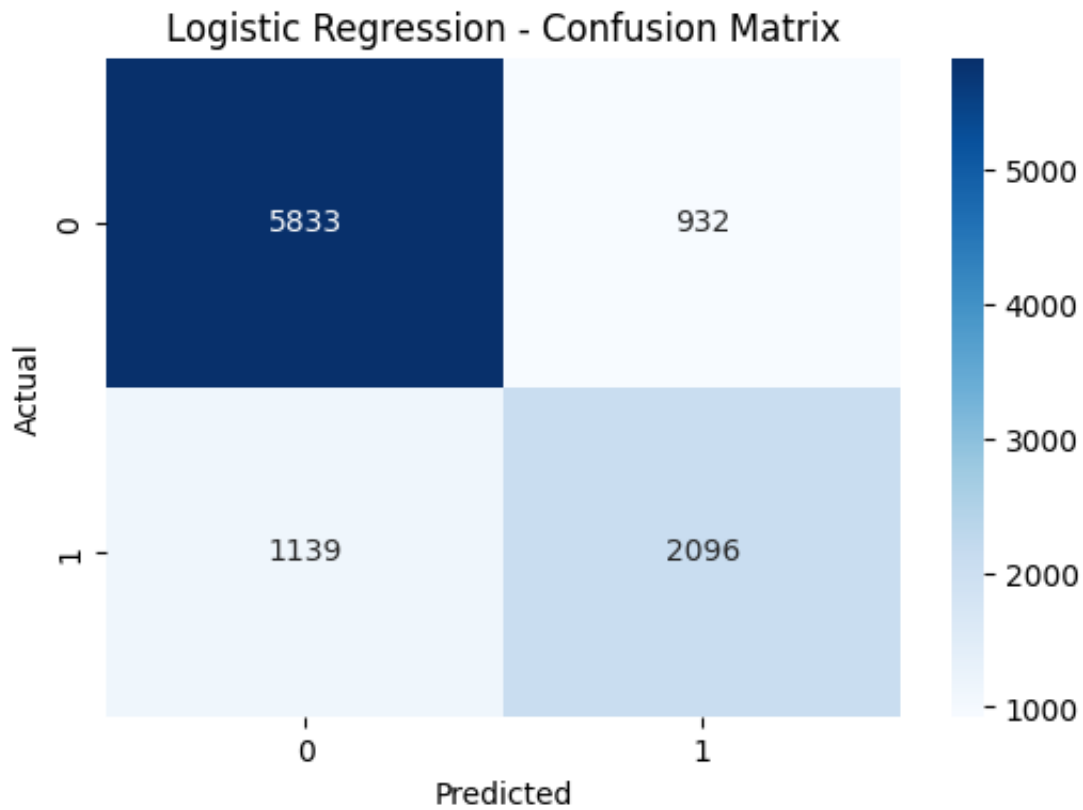
```

[[5833  932]
 [1139 2096]]

```

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.86	0.85	6765
1	0.69	0.65	0.67	3235
accuracy			0.79	10000
macro avg	0.76	0.76	0.76	10000
weighted avg	0.79	0.79	0.79	10000



```
In [1]: full_encoded_df.to_csv('fraud_detection_processed.csv', index=False)
```

```
-----
-----
NameError                                Traceback (most recent call l
ast)
Cell In[1], line 1
----> 1 full_encoded_df.to_csv('fraud_detection_processed.csv', index=F
alse)

NameError: name 'full_encoded_df' is not defined
```

```
In [ ]:
```